1. High-Level Summary of Functionality

This codebase implements an Android application (**PDR & 6-State EKF Diagnostic**) built with Kotlin and Jetpack Compose. Its primary purpose is to provide precise pedestrian navigation and tracking by fusing **Pedestrian Dead Reckoning (PDR)** sensor data (accelerometer, gyroscope, and rotation vector) with periodic **GPS fixes** using a **6-State Extended Kalman Filter (EKF)**.

Key features include:

- **Sensor Fusion & EKF:** Continuously tracks position, velocity, stride length, and compass bias.
- **Route & Leg Navigation:** Parses hiking routes (supporting Irish National Grid coordinates via CSV), tracks progress across route legs, detects off-course drifting, and issues contextual alerts.
- **Text-to-Speech (TTS):** Automatically vocalizes navigation announcements and warnings.
- **Telemetry Recording & Playback:** Records live hiking telemetry sessions to CSV and features a full playback engine with stepping, speed scaling, and deterministic replaying.
- **Interactive Diagnostic UI:** Includes live telemetry dashboards, customizable map renderers with grid overlays and "Leg View" rotation, and control panels.

2. Purpose of Classes and Components

Hike & Navigation (`com.example.pdrandroutetest.hike`)

- `Hike` : Manages active hike progression, evaluates leg states (e.g., target reached, approaching, off-course), and generates text instructions.
- `LegAnnouncementTracker` : Flags whether specific journey milestones (leg start, approach, arrival, hike completion) have been announced for the current leg.
- `NavigationMessage` : Data class storing timestamped navigation messages linked to breadcrumb indices.

Sensor, EKF & Data (`sensor` , `ekf` , `data`)

- `DiagnosticEkf` : Implements the core 6-state Extended Kalman Filter tracking position (x, y), velocity (v_x, v_y), stride length (L), and compass bias (b_θ).
- `EkfConfig` : Stores configuration parameters such as process noise covariances and stride length limits.

- `PdrStepDetector` : Listens to hardware sensors (`TYPE_ACCELEROMETER` , `TYPE_ROTATION_VECTOR` , `TYPE_GYROSCOPE`) to detect step peaks and compute heading angles.
- `StepDetectorConfig` : Defines detection thresholds, minimum time intervals, and low-pass filter smoothing factors.
- `EkfState` , `Estimate` , `Vector2D` : Data structures representing filter states, estimated values, and associated error variances.

Telemetry & Playback (`com.example.pdrandroutetest.telemetry`)

- `TelemetryPlaybackEngine` : Manages asynchronous playback, pausing, speed multipliers, and step-forward/step-back rewinding of recorded telemetry streams.
- `TelemetryEvent` : Sealed class defining discrete recorded events (`GpsFixEvent` , `StepDetectedEvent` , `PredictionTickEvent`).
- `TelemetryRecorder` : Records telemetry events and serializes/deserializes them to and from CSV format.

Models (`com.example.pdrandroutetest.model`)

- `Route` , `Leg` , `Waypoint` : Represent structured hiking routes composed of sequential legs and target waypoints with arrival/approach thresholds.
- `Breadcrumb` : Records historical location points along with timestamps, GPS flags, and EKF state snapshots.
- `Location` : Base coordinate class providing Euclidean distance and bearing calculation methods.

UI & View Model (`com.example.pdrandroutetest.ui`)

- `DiagnosticViewModel` : Acts as the central ViewModel exposing reactive `StateFlow` s for UI components, managing playback controls, and initializing TTS.
- `MapScreen` : Custom Jetpack Compose canvas renderer that draws routes, waypoints, approach/arrival radii, breadcrumb trails, grid lines, and hiker heading indicators.
- `ControlPanel` : UI component for toggling tracking, selecting GPS polling intervals, and controlling track playback.
- `TelemetryDashboard` : Displays real-time EKF telemetry metrics (position, bearing, speed, stride length, compass bias).
- `NavigationMessagePanel` : Displays recent navigation text notifications.

Services & Repositories (`service` , `repository` , `util`)

- `DiagnosticForegroundService` : Android Foreground Service that runs location updates, step detection callbacks, and 1 Hz EKF prediction ticks in the background.
- `HikeRepository` : Central repository managing reactive shared flows for EKF states, breadcrumb histories, and route configuration.
- `CoordinateUtils` & `IrishGridConverter` : Handles geodetic conversions between WGS84 and Irish National Grid coordinates, distance, and bearing math.
- `RouteParser` : Parses CSV files into structured waypoints and routes.
- `TtsManager` : Manages Android Text-to-Speech synthesis and playback.

3. Summary of Non-Trivial Methods

- `DiagnosticEkf.onGpsFix(...)`
  - *Function:* Fuses external GPS coordinate measurements into the EKF. On the first fix, it initializes position and variances. On subsequent fixes, it computes longitudinal and latitudinal residuals relative to current EKF estimates and applies scalar measurement updates (`updateScalarObservation`) to correct position, stride length, and compass bias.
- `DiagnosticEkf.predict(dtSec)`
  - *Function:* Performs the EKF time-update prediction step. It advances position based on current velocity vectors over time interval `dtSec` , computes the state transition Jacobian matrix, propagates error covariance matrix P, and adds process noise (`qDiag`).
- `Hike.evaluateLegState(...)`
  - *Function:* Evaluates hierarchical navigation rules for the current hiker location. It checks conditions in strict priority order: target arrival, hike completion, overshooting targets (`TOO_FAR`), entering approach windows, crossing leg starts, and off-course drift with cool-off timer management.
- `TelemetryPlaybackEngine.stepBack()`
  - *Function:* Rewinds playback by one event. Because EKF state depends on history, it resets the EKF and hike state, then deterministically replays all recorded telemetry events from index 0 up to the target index minus one.
- `IrishGridConverter.Wgs84ToIrishGrid(...)` / `IrishGridToWgs84(...)`
  - *Function:* Executes complex transverse Mercator projections, meridional arc calculations, and local grid shift corrections to convert between WGS84 latitude/longitude and alphanumeric Irish National Grid coordinates.

- `MapScreen` **(Canvas Drawing &** `ViewportTracker.updateAll` **)**
  - *Function:* Computes dynamic bounding boxes, scaling factors, and coordinate transformations (including specialized "Leg View" rotation that aligns the active leg vertically) to render custom vector graphics, grid lines, and position arrows efficiently within UI frame budgets.
- `PdrStepDetector.onSensorChanged(...)`
  - *Function:* Processes high-frequency accelerometer and rotation vector sensor events. It applies a low-pass filter to acceleration magnitudes, evaluates step thresholds and minimum time intervals to detect steps, and records synchronized heading angles.